CHARUSAT
CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY

FACULTY OF TECHNOLOGY AND ENGINEERING

DEVANG PATEL INSTITUE OF ADVANCE TECHNOLOGY AND RESEARCH

DEPARTMENT OF INFORMATION TECHNOLOGY

A.Y. 2026-27 Semester-3

LAB MANUAL

CSUC201 Fundamentals of Data Structures and Algorithms

DEPSTAR

Practical-10

Practical AIM:

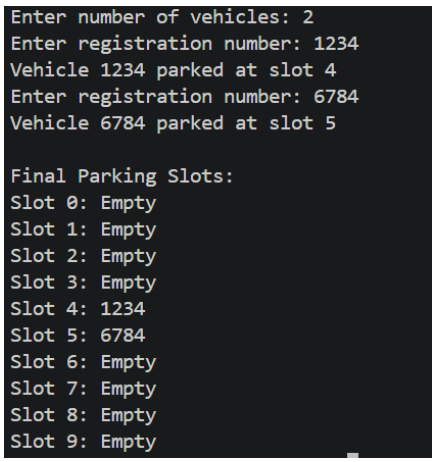
To implement hash tables using different collision handling strategies and develop the ability to reason about why collisions occur, how different strategies resolve them, and what trade-offs each approach involves.

10.1

Problem Defination:

A small parking lot has exactly 10 numbered slots (0 to 9). When a vehicle arrives, its registration number is used to assign it a slot by taking the last digit of the number. If that slot is already taken, the vehicle moves to the next available slot in order, wrapping around if necessary. Given a sequence of vehicle registrations, assign each to a slot using this rule and display the final state of all ten slots. Describe what your solution does when it encounters an already occupied slot. What happens when the parking lot is nearly full and many vehicles keep landing on the same slot? Can you think of a situation where your solution might loop indefinitely?

Output:



Difficulty Faced:

Handling collisions when the assigned parking slot was already occupied and managing wrap-around at slot 9.

Solution:

Used linear probing to check the next available slot, wrapping back to slot 0 when necessary. Checked whether all slots were full to prevent an infinite loop.

Outcome:

Successfully assigned vehicles to available slots and displayed the final parking lot state without infinite looping.

10.2

Problem Defination:

A library uses a system with 10 shelves numbered 0 to 9. Each book's code is used to decide which shelf it belongs to by taking the code modulo 10. Unlike the parking lot, each shelf can hold multiple books stacked together — when two books map to the same shelf, they are

simply added to that shelf's stack. Given a sequence of book codes, place each on the correct shelf and display the final contents of all shelves. Describe how your solution handles two books that map to the same shelf. What happens to search time on a shelf as more and more books pile onto it? Can you think of what would cause all books to end up on the same shelf?

Output:

```
Enter number of books: 3
Enter book code: 768
Book 768 placed on shelf 8
Enter book code: 932
Book 932 placed on shelf 2
Enter book code: 402
Book 402 placed on shelf 2

Final Shelf Contents:
Shelf 0: Empty
Shelf 1: Empty
Shelf 2: 932 -> 402 -> NULL
Shelf 3: Empty
Shelf 4: Empty
Shelf 5: Empty
Shelf 6: Empty
Shelf 7: Empty
Shelf 8: 768 -> NULL
Shelf 9: Empty
```

Difficulty Faced:

Handling multiple books mapped to the same shelf and managing each shelf’s stack.

Solution:

Used separate chaining to store multiple books on the same shelf and applied modulo 10 to determine the shelf index.

Outcome:

Successfully stored and displayed all books. Search time increases as more books accumulate on one shelf. If all book codes have the same remainder modulo 10, all books end up on the same shelf.

10.3

Problem Defination:

A university assigns student IDs to storage slots using a two-step rule. The first rule determines the initial slot. If that slot is taken, instead of simply moving one slot forward, a second rule based on the ID itself determines how many slots to jump. This means different IDs that collide will probe different sequences of slots, spreading them more evenly. Given a sequence of student IDs, assign each to a slot using both rules and display the final state of the table. Describe how your solution decides how many slots to jump when a collision occurs. What is the risk if the jump size happens to be a factor of the table size? How does this strategy differ from Problem 10.1 in how it spreads collisions?

Output:

```
Enter number of students: 2
Enter student ID: 2783
Student 2783 placed at slot 3
Enter student ID: 4532
Student 4532 placed at slot 2

Final Hash Table:
Slot 0: Empty
Slot 1: Empty
Slot 2: 4532
Slot 3: 2783
Slot 4: Empty
Slot 5: Empty
Slot 6: Empty
Slot 7: Empty
Slot 8: Empty
Slot 9: Empty
```

Difficulty Faced:

Choosing a suitable jump size during collisions and ensuring all table slots can be checked.

Solution:

Used double hashing, where a second hash function determines the jump size whenever a collision occurs.

Outcome:

Collisions were distributed more evenly than linear probing. If the jump size and table size have a common factor, some slots may never be visited.

Key Questions:

1.If five consecutive vehicle registrations all end in the same digit, trace where each one gets placed in the parking lot. Now explain why this clustering effect is a structural weakness of this collision strategy — not just for this input, but in general.

Sol: All five vehicles initially map to the same slot. They occupy that slot and the next four available slots using linear probing. This causes primary clustering, where groups of occupied slots grow and future collisions become more frequent.

2.Suppose all book codes in Problem 2 are multiples of 10. Without tracing through the code, predict what the final shelf layout would look like and explain why. What does this tell you about the relationship between the hash function and the input data?

Sol: All books will be placed on shelf 0 because multiples of 10 give a remainder of 0 when divided by 10. This shows that a poor match between the hash function and input data can cause uneven distribution.

3.In Problem 3, what would happen if the second rule always produced a jump size of zero?

Sol:Trace through the consequence and explain how this failure is related to the choice of the second hash function rather than the data itself. If the second hash function always returns a jump size of zero, the vehicle repeatedly checks the same occupied slot, causing an infinite loop. This is a flaw in the second hash function because the jump size must be non-zero.

Github Link:

https://github.com/brindajivani/DSA/tree/main/Practical10